

NUST Service Centre Simulation Designing and Evaluating Data Structures and Algorithms

Group Mini-Project 2026 — Project Report

GROUP 11

#	Full Name	Student Number	Role
1	George Shimaneni	225018497	Group leader
2	Dipundhi Paul Peter	225058146	Member
3	Mangulukeni Kayoko	227074404	Member
4	Nelumbu Rachel	225049511	Member
5	Enerist T Shilumbu	222093951	Member

Item	Detail
Programming language	Java (JDK 8 or newer), console based
Submitted by	225018497 – George Shimaneni
Submission date	25 September 2026
GitHub repository	https://github.com/georgeshimaneni23-beep/DSA521S-Group11-ServiceCentre
Declaration	All data structures and sorting algorithms in this project were implemented by the group. No built-in Java collection or sorting method was used as a replacement. Every member understands and can defend the submitted solution.

1. Introduction and System Overview

This report documents Group 11's solution to the DSA521S Campus Service Centre simulation. The program models a normal day at the NUST student service centre: students arrive for registration, fee enquiries, student-card replacement, academic enquiries and document collection, they wait in a single line, they are assisted in arrival order, a permanent service record is kept for each of them, and the day's service times are then analysed and sorted.

The emphasis is not on a user interface. Every data structure and every sorting algorithm is implemented from first principles so that the behaviour, the cost and the suitability of each one can be demonstrated and measured.

Structure of the solution

Part	Requirement	Structure / algorithm	Java file
A1	Waiting line	Queue (FIFO, linked nodes)	StudentQueue.java
A2	Student service records	Singly linked list	ServiceRecordList.java
A3	Postfix evaluation	Stack (LIFO, array based)	IntStack.java, PostfixEvaluator.java
A4	Daily statistics	Array with manual traversal	DailyStatistics.java
B	Four sorting algorithms	Selection, Insertion, Merge, Quick	Sorters.java
C	Algorithm experiment	Comparison counting and timing	SortingExperiment.java
D	Integrated system	Menu driving all of the above	ServiceCentreApp.java
—	Shared data and helpers	Student record, array helpers	Student.java, ArrayUtil.java
—	Automatic demonstration	Prints every required trace	DemoRunner.java

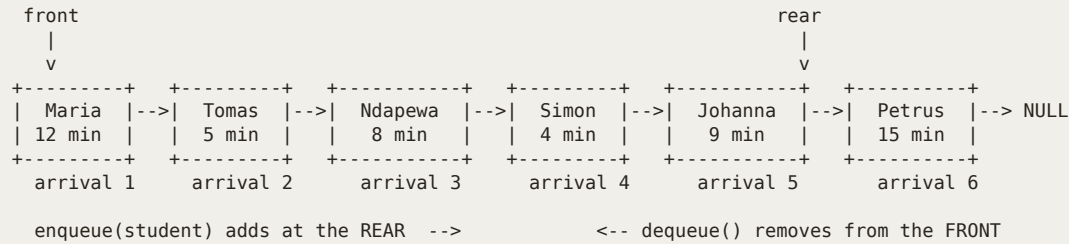
How to run: `javac -d out src/*.java` then `java -cp out ServiceCentreApp` for the interactive menu, or `java -cp out DemoRunner` to print every demonstration and trace shown in this report.

2. Part A — Data Structure Design

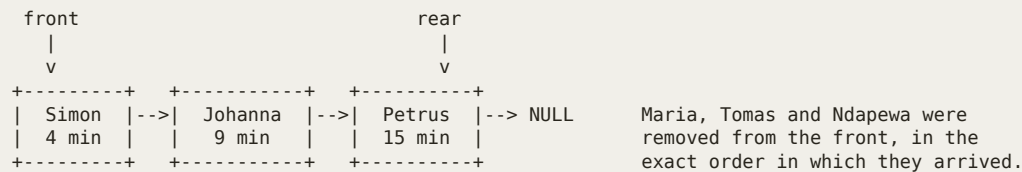
2.1 Task A1 — Waiting Line (Queue)

The waiting line is a queue of Student objects built from a chain of internal nodes with a front pointer (where students leave) and a rear pointer (where students join). Because both ends are tracked, enqueue() and dequeue() are both O(1) no matter how long the line becomes.

Queue diagram — after six arrivals



Queue diagram — after three students have been served



Pseudocode for enqueue() and dequeue() is given in section 5.1.

```
Task A1 - Six arrivals and three students served

=====
TASK A1 - WAITING LINE (QUEUE)
=====

SIX ARRIVALS (enqueue):
arrival 1: enqueue(Maria(221045678))
arrival 2: enqueue(Tomas(222034512))
arrival 3: enqueue(Ndapewa(223041876))
arrival 4: enqueue(Simon(221067341))
arrival 5: enqueue(Johanna(224011290))
arrival 6: enqueue(Petrus(223098455))

isEmpty() = false
peek()    = Maria(221045678) (first to be served)

--- WAITING QUEUE (front to rear) ---
Pos  StudentNo  Name      Service      Est.Time
1    221045678  Maria     Registration  12 min
2    222034512  Tomas     Student Card  5 min
3    223041876  Ndapewa   Fees          8 min
4    221067341  Simon     Documents     4 min
5    224011290  Johanna   Academic Enquiry 9 min
6    223098455  Petrus    Document Collect 15 min
Front of queue : Maria(221045678)
Rear of queue  : Petrus(223098455)
Students waiting: 6

Queue diagram after 6 arrivals:
FRONT -> [Maria] -> [Tomas] -> [Ndapewa] -> [Simon] -> [Johanna] -> [Petrus] -> REAR

THREE STUDENTS SERVED (dequeue):
service 1: dequeue() removed Maria(221045678) - Registration (12 min)
           queue now: FRONT -> [Tomas] -> [Ndapewa] -> [Simon] -> [Johanna] -> [Petrus] -> REAR
service 2: dequeue() removed Tomas(222034512) - Student Card (5 min)
           queue now: FRONT -> [Ndapewa] -> [Simon] -> [Johanna] -> [Petrus] -> REAR
service 3: dequeue() removed Ndapewa(223041876) - Fees (8 min)
           queue now: FRONT -> [Simon] -> [Johanna] -> [Petrus] -> REAR

--- WAITING QUEUE (front to rear) ---
Pos  StudentNo  Name      Service      Est.Time
1    221067341  Simon     Documents     4 min
2    224011290  Johanna   Academic Enquiry 9 min
3    223098455  Petrus    Document Collect 15 min
Front of queue : Simon(221067341)
Rear of queue  : Petrus(223098455)
Students waiting: 3

isEmpty() = false, peek() = Simon(221067341)
```

Figure 1 — Task A1: six arrivals (enqueue), peek(), isEmpty(), displayQueue() and three students served (dequeue).

Why a queue is appropriate

Service-centre policy is that students are assisted in the order in which they arrived. A queue enforces that rule structurally: values can only be added at the rear and can only be removed from the front, so the student who has waited longest is always the next one served and no code path can accidentally jump the line. An array would need every remaining element shifted forward each time a student is served ($O(n)$ per service), and a stack would serve the most recent arrival first, which is precisely the wrong behaviour.

2.2 Task A2 — Student Service Records (Singly Linked List)

Each node stores a data part (student number, name, service type, estimated service time) and a next pointer. The list supports insertion at the beginning, at the end and at a specified position, deletion by student number, linear search and full traversal. This same list and these same operations are used by menu options 4 to 7 of the integrated system in Part D.

Link change 1 — insertion at position 3 (Johanna)

```
BEFORE
head
|
v
+-----+ +-----+ +-----+ +-----+
| Simon |---->| Maria |---->| Tomas |---->| Ndapewa |----> NULL
+-----+ +-----+ +-----+ +-----+
node 1      node 2      node 3      node 4

AFTER insertStudent(Johanna, position 3)
head
|
v
+-----+ +-----+ +-----+ +-----+ +-----+
| Simon |---->| Maria |---->| Johanna |---->| Tomas |---->| Ndapewa |----> NULL
+-----+ +-----+ +-----+ +-----+ +-----+
node 1      node 2      NEW node 3      node 4      node 5

Two links change: (1) Johanna.next <- the node that followed Maria (Tomas)
                  (2) Maria.next   <- Johanna
No other node moves in memory, and nothing is shifted.
```

Link change 2 — deletion of Tomas (222034512)

```
BEFORE
head
|
v
+-----+ +-----+ +-----+ +-----+ +-----+
| Simon |---->| Maria |---->| Johanna |---->| Tomas |---->| Ndapewa |----> NULL
+-----+ +-----+ +-----+ +-----+ +-----+
                                previous      target

AFTER deleteStudent("222034512")
head
|
v
+-----+ +-----+ +-----+ +-----+
| Simon |---->| Maria |---->| Johanna |---->| Ndapewa |----> NULL
+-----+ +-----+ +-----+ +-----+
                                \
                                ---> (Tomas node is bypassed and is no
                                      longer reachable from head)

One link changes: previous.next <- target.next
```

```
Task A2 - Linked list before and after link changes

=====
TASK A2 - STUDENT SERVICE RECORDS (SINGLY LINKED LIST)
=====

1) insertStudent at the END (building the list):
after insertAtEnd(Maria) : HEAD -> [Maria | next] -> NULL
after insertAtEnd(Tomas) : HEAD -> [Maria | next] -> [Tomas | next] -> NULL
after insertAtEnd(Ndapewa) : HEAD -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> NULL

2) insertStudent at the BEGINNING
LIST BEFORE : HEAD -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> NULL
LIST AFTER : HEAD -> [Simon | next] -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> NULL
Link change : new node Simon.next now points to the old head (Maria),
              and head now points to Simon.

3) insertStudent at a SPECIFIED POSITION (position 3)
LIST BEFORE : HEAD -> [Simon | next] -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> NULL
LIST AFTER : HEAD -> [Simon | next] -> [Maria | next] -> [Johanna | next] -> [Tomas | next] -> [Ndapewa | next] -> NULL
Link change : node 2 (Maria).next now points to Johanna, and
              Johanna.next points to the node that followed Maria (Tomas).
```

Figure 2 — Task A2: insertion at the beginning, at the end and at position 3, showing the list before and after each link change.

Insertion at the beginning is $O(1)$; insertion at the end or at position k , deletion and search all require walking the chain, so they are $O(n)$ in the worst case. The search in the demonstration run found student 223041876 at node 5 after 5 comparisons, and reported 5 comparisons before concluding that 999999999 is absent — the expected behaviour of a sequential search on an unsorted list.

2.3 Task A3 — Postfix Expression Evaluation (Stack)

This is an independent exercise and is deliberately not part of the service-centre menu. The stack is array-based and grows by copying into a larger array when it becomes full. Operands are pushed; when an operator is read, the top two values are popped (the second operand is popped first), the operation is applied, and the result is pushed back. The operators $+$, $-$, $\times$ and $\div$ are supported.

Trace of $5\ 3\ +\ 2\ *$

Step	Token	Action	Stack (bottom $\rightarrow$ top)
1	5	push 5	[5]
2	3	push 3	[5, 3]
3	+	pop 3, pop 5, push $5 + 3 = 8$	[8]
4	2	push 2	[8, 2]
5	*	pop 2, pop 8, push $8 \times 2 = 16$	[16]
6	—	peek() reads the answer, then pop()	[] $\rightarrow$ result = 16

```
Task A3 - Postfix evaluation using a Stack

=====
TASK A3 - POSTFIX EXPRESSION EVALUATION USING A STACK
=====

Evaluating postfix expression: 5 3 + 2 *

Token  Action                                Stack (bottom -> top)
-----
5      push 5                                [ 5 ] <- top
3      push 3                                [ 5, 3 ] <- top
+      pop 3, pop 5, push 5+3=8              [ 8 ] <- top
2      push 2                                [ 8, 2 ] <- top
*      pop 2, pop 8, push 8*2=16             [ 16 ] <- top
-----

peek() on the stack gives the final result = 16
Result of "5 3 + 2 *" = 16 (expected 16)

Further checks (no trace printed):
"12 5 8 4 - * +" = 32
"20 4 / 3 +" = 8
"7 2 - 3 *" = 15
```

Figure 3 — Task A3: the stack contents printed after every operation, with three further expressions verified.

Why a stack is appropriate

In postfix notation the operands an operator needs are always the two most recently produced values, and results feed straight back into later operators. That is exactly last-in-first-out behaviour, so push() and pop() supply the correct operands with no searching and no index arithmetic. The whole expression is evaluated in a single $O(n)$ pass.

2.4 Task A4 — Daily Statistics (Array)

Every time a student is served, the service time is appended to a plain int array (the array is grown manually by copying into a larger one when it fills up). All six statistics are then produced by traversing that array with ordinary loops. No built-in max(), min(), sum(), average or stream helper is used.

Worked example — array [12, 5, 8, 4, 9, 15, 3, 11]

Statistic	Method of calculation	Result
Total students served	counter incremented once per array position	8
Total service time	running total accumulated over one pass	67 minutes
Average service time	total ÷ number of students	8.38 minutes
Highest service time	first element kept, replaced whenever a larger value is met	15 minutes
Lowest service time	first element kept, replaced whenever a smaller value is met	3 minutes
Services longer than 10 min	counter incremented when element > 10 (12, 15, 11)	3

```
Options 2 and 8 - Queue dequeue and Array statistics

Select option: 2

Next in line (peek): Maria(221045678)
Now serving: 221045678 Maria Registration 12 min
Service time recorded in the daily statistics array.
Queue diagram: FRONT -> [Tomas] -> [Ndapewa] -> [Simon] -> REAR

Select option: 8

--- DAILY STATISTICS (computed from the service-time array) ---
Array contents : [12, 5]
Total students served : 2
Total service time : 17 minutes
Average service time : 8.50 minutes
Highest service time : 12 minutes
Lowest service time : 5 minutes
Services over 10 min : 1
```

Figure 4 — menu option 2 (serving a student, whose service time is recorded in the array) followed by menu option 8 (the array statistics).

Why an array is appropriate

The statistics are computed over a fixed set of completed service times: values are only appended at the end and read back in order, never inserted in the middle. An array stores those values contiguously with $O(1)$ access by index, so a single traversal computes all six figures, and no memory is wasted on pointers. It is also the natural input form for the sorting algorithms in Part B, which need direct index access to swap values in place — something a singly linked list cannot provide efficiently.

2.5 Task A5 — Data-Structure Justification

Structure	Assigned task	Why its behaviour fits this task
Queue	Waiting line (A1)	Students must be assisted in arrival order. FIFO removal from the front and insertion at the rear make that policy part of the structure itself, so the order cannot be broken by a coding mistake. Both operations are $O(1)$, so the program does not slow down as the line grows, whereas serving from the front of an array would cost $O(n)$ in shifting every time.
Singly linked list	Student service records (A2)	The number of records for a day is unknown in advance and records are inserted at chosen positions and removed from the middle. A linked list allocates one node at a time so it never has to be resized, and an insertion or deletion only re-points one or two next references instead of shifting every later element as an array would. Only a forward traversal is ever needed, so the extra memory of a doubly linked list would not be justified.
Stack	Postfix evaluation (A3)	The operands required by an operator are always the two most recently produced values, and each result is immediately available to the next operator. That is last-in-first-out access, so <code>push()</code> and <code>pop()</code> hand back exactly the right operands with no searching, letting the expression be evaluated in one $O(n)$ pass.
Array	Daily statistics (A4)	The service times form a fixed, index-addressable block that is only appended to and then read in order. Contiguous storage with $O(1)$ index access lets a single loop compute the total, average, highest, lowest and over-10-minute count, and gives the sorting algorithms the in-place element swapping they depend on.

3. Part B — Sorting Algorithm Challenge

All four algorithms sort the array given in the brief, [17, 5, 23, 8, 14, 3, 11, 20, 6, 9], into ascending order. Each one produced [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]. Only comparisons between data values are counted.

Algorithm	Comparisons	Movements	Movement type
Selection Sort	45	7	swaps
Insertion Sort	32	25	shifts
Merge Sort	23	34	merge moves
Quick Sort	20	12	swaps

3.1 Task B1 — Selection Sort

Each pass scans the unsorted part of the array, finds the smallest remaining value and swaps it into the next sorted position. Totals for this array: 45 comparisons (always $n(n-1)/2 = 45$) and 7 swaps.

Pass	Smallest found	Array state after the pass	Comparisons so far	Swaps so far
Start	—	[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]	0	0
1	3	[3 5, 23, 8, 14, 17, 11, 20, 6, 9]	9	1
2	5	[3, 5 23, 8, 14, 17, 11, 20, 6, 9]	17	1
3	6	[3, 5, 6 8, 14, 17, 11, 20, 23, 9]	24	2

The bar marks the boundary between the sorted prefix and the unsorted remainder. In pass 2 the value 5 was already in the correct position, so no swap was performed — but the 8 comparisons of that pass still had to be made, which is why Selection Sort cannot benefit from partially sorted data.

3.2 Task B2 — Insertion Sort

Each pass takes the next value (the key), shifts every larger value in the sorted prefix one place to the right, and drops the key into the gap. Totals for this array: 32 comparisons and 25 shifts.

Pass	Key	Array state after the pass	Shifts in pass	Comparisons so far	Shifts so far
Start	—	[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]	—	0	0
1	5	[5, 17 23, 8, 14, 3, 11, 20, 6, 9]	1	1	1
2	23	[5, 17, 23 8, 14, 3, 11, 20, 6, 9]	0	2	1
3	8	[5, 8, 17, 23 14, 3, 11, 20, 6, 9]	2	5	3

Pass 2 is the informative one: 23 is already larger than everything before it, so a single comparison ends the pass with no shifting. This early exit is what gives Insertion Sort its $O(n)$ best case on sorted and almost-sorted data, confirmed experimentally in Part C.

```
=====
TASK B1 - SELECTION SORT
=====
Initial array : [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
Pass 1: smallest in unsorted part = 3, swapped into index 0
      array = [3 | 5, 23, 8, 14, 17, 11, 20, 6, 9] (values left of | are sorted)
      running totals: comparisons = 9, swaps = 1
Pass 2: smallest in unsorted part = 5, already in place
      array = [3, 5 | 23, 8, 14, 17, 11, 20, 6, 9] (values left of | are sorted)
      running totals: comparisons = 17, swaps = 1
Pass 3: smallest in unsorted part = 6, swapped into index 2
      array = [3, 5, 6 | 8, 14, 17, 11, 20, 23, 9] (values left of | are sorted)
      running totals: comparisons = 24, swaps = 2
Sorted array  : [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
TOTAL comparisons = 45, TOTAL swaps = 7

=====
TASK B2 - INSERTION SORT
=====
Initial array : [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
Pass 1: key = 5 inserted at index 0 (1 shift(s) in this pass)
      array = [5, 17 | 23, 8, 14, 3, 11, 20, 6, 9] (values left of | are sorted)
      running totals: comparisons = 1, shifts = 1
Pass 2: key = 23 inserted at index 2 (0 shift(s) in this pass)
      array = [5, 17, 23 | 8, 14, 3, 11, 20, 6, 9] (values left of | are sorted)
      running totals: comparisons = 2, shifts = 1
Pass 3: key = 8 inserted at index 1 (2 shift(s) in this pass)
      array = [5, 8, 17, 23 | 14, 3, 11, 20, 6, 9] (values left of | are sorted)
      running totals: comparisons = 5, shifts = 3
Sorted array  : [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
TOTAL comparisons = 32, TOTAL shifts = 25
```

Figure 5 — Tasks B1 and B2: program output showing the first three passes and the final comparison and movement counts.

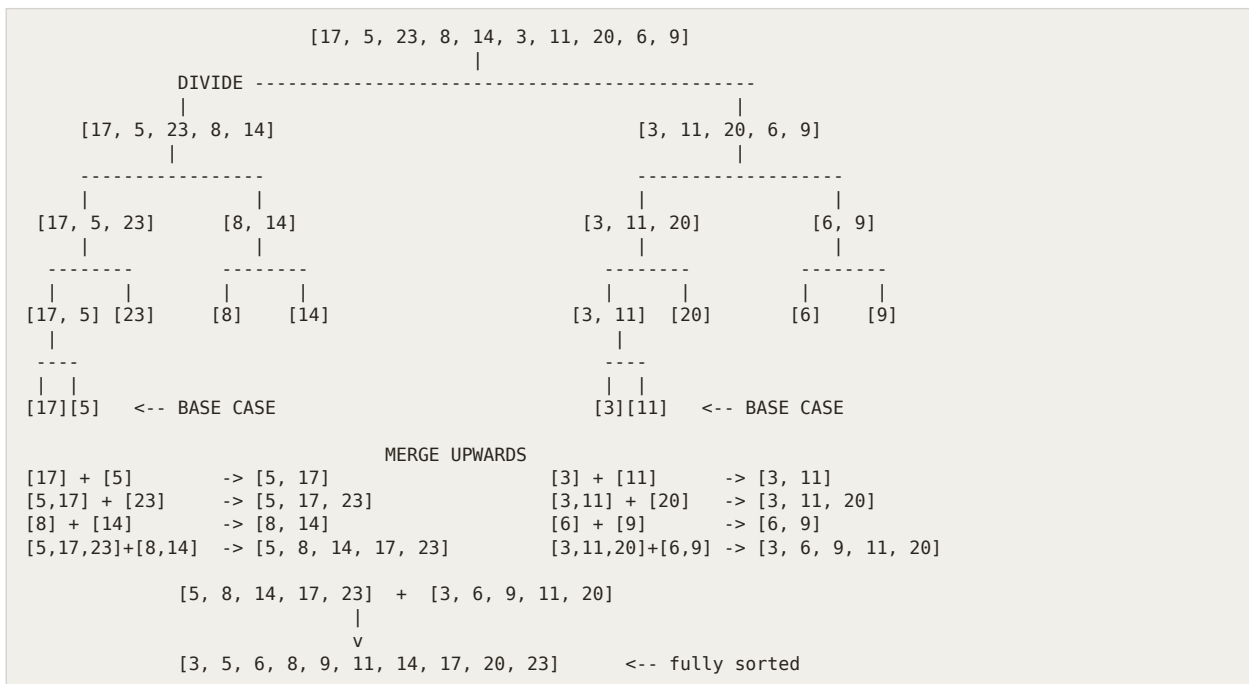
3.3 Task B3 — Merge Sort

Merge Sort repeatedly divides the array in half until each piece holds a single element, then merges the sorted pieces back together in order. Totals: 23 comparisons and 34 merge moves.

Base case

A sub-array of length 0 or 1 is already sorted, so the recursion stops and returns immediately (in code: `if (low >= high) return;`). Every branch of the tree below ends in such a single-element array.

Complete divide-and-merge process



3.4 Task B4 — Quick Sort

Pivot-selection rule: the last element of the current sub-array (Lomuto partitioning). Values less than or equal to the pivot are moved to the left of it, larger values stay to its right, and the pivot is then placed in its final position between the two partitions. Totals: 20 comparisons and 12 swaps.

Stage	Sub-array (indices)	Pivot	Left partition ($\leq$ pivot)	Right partition ($>$ pivot)	Array after the stage
1	[17, 5, 23, 8, 14, 3, 11, 20, 6, 9] (0–9)	9 (last)	[5, 8, 3, 6]	[23, 11, 20, 17, 14]	[5, 8, 3, 6, 9, 23, 11, 20, 17, 14]
2	[5, 8, 3, 6] (0–3)	6 (last)	[5, 3]	[8]	[5, 3, 6, 8, 9, 23, 11, 20, 17, 14]

After stage 1 the pivot 9 is at index 4 and will never move again; the algorithm then recurses into the left partition (indices 0–3) and later the right partition (indices 5–9). The base case is again a sub-array of 0 or 1 element.

```

Tasks B3 and B4 - Merge Sort and Quick Sort traces
=====
TASK B3 - MERGE SORT
=====
Initial array : [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

DIVIDE and MERGE trace (indentation shows the recursion depth):
DIVIDE [17, 5, 23, 8, 14, 3, 11, 20, 6, 9] into [17, 5, 23, 8, 14] and [3, 11, 20, 6, 9]
  DIVIDE [17, 5, 23, 8, 14] into [17, 5, 23] and [8, 14]
    DIVIDE [17, 5, 23] into [17, 5] and [23]
      DIVIDE [17, 5] into [17] and [5]
        BASE CASE: [17] - a single element is already sorted.
        BASE CASE: [5] - a single element is already sorted.
        MERGE -> [5, 17]
      BASE CASE: [23] - a single element is already sorted.
      MERGE -> [5, 17, 23]
    DIVIDE [8, 14] into [8] and [14]
      BASE CASE: [8] - a single element is already sorted.
      BASE CASE: [14] - a single element is already sorted.
      MERGE -> [8, 14]
    MERGE -> [5, 8, 14, 17, 23]
  DIVIDE [3, 11, 20, 6, 9] into [3, 11, 20] and [6, 9]
    DIVIDE [3, 11, 20] into [3, 11] and [20]
      DIVIDE [3, 11] into [3] and [11]
        BASE CASE: [3] - a single element is already sorted.
        BASE CASE: [11] - a single element is already sorted.
        MERGE -> [3, 11]
      BASE CASE: [20] - a single element is already sorted.
      MERGE -> [3, 11, 20]
    DIVIDE [6, 9] into [6] and [9]
      BASE CASE: [6] - a single element is already sorted.
      BASE CASE: [9] - a single element is already sorted.
      MERGE -> [6, 9]
    MERGE -> [3, 6, 9, 11, 20]
  MERGE -> [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

Sorted array : [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
TOTAL comparisons = 23, TOTAL merge moves = 34
=====

TASK B4 - QUICK SORT
=====
Initial array : [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
Pivot rule : the LAST element of the sub-array (Lomuto partitioning).

Partition stage 1 (sub-array indices 0..9)
  sub-array before : [17, 5, 23, 8, 14, 3, 11, 20, 6, 9]
  pivot : 9 (last element)
  left partition : [5, 8, 3, 6] (values <= pivot)
  pivot final pos : index 4
  right partition : [23, 11, 20, 17, 14] (values > pivot)
  whole array now : [5, 8, 3, 6, 9, 23, 11, 20, 17, 14]

Partition stage 2 (sub-array indices 0..3)
  sub-array before : [5, 8, 3, 6]
  pivot : 6 (last element)
  left partition : [5, 3] (values <= pivot)
  pivot final pos : index 2
  right partition : [8] (values > pivot)
  whole array now : [5, 3, 6, 8, 9, 23, 11, 20, 17, 14]

Sorted array : [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
TOTAL comparisons = 20, TOTAL swaps = 12

```

Figure 6 — Tasks B3 and B4: the merge-sort divide/merge trace with the base cases marked, and the first two Quick Sort partitioning stages.

4. Part C — Algorithm Experiment

4.1 Method

- Test values are generated programmatically in the range 1–999 using a reproducible generator with a fixed seed (2026), so the tables below can be reproduced exactly.
- For each input size one original array is generated, and every algorithm is given its own copy of exactly the same values, so the comparison is fair.
- Only comparisons between data values are counted; loop-index and boundary tests are not.
- `System.nanoTime()` is read immediately before and immediately after the sorting call only, so array generation, copying and printing are excluded from the measured time.

4.2 Results — random arrays

Algorithm	Input Size	Number of Comparisons	Execution Time (ns)
Selection Sort	20	190	7 919
Insertion Sort	20	90	6 974
Merge Sort	20	68	13 299
Quick Sort	20	81	7 211
Selection Sort	50	1 225	30 513
Insertion Sort	50	652	19 446
Merge Sort	50	221	31 005
Quick Sort	50	271	14 852
Selection Sort	100	4 950	109 052
Insertion Sort	100	2 554	73 402
Merge Sort	100	548	65 349
Quick Sort	100	705	31 024
Selection Sort	500	124 750	2 589 246
Insertion Sort	500	61 625	1 704 529
Merge Sort	500	3 861	403 080
Quick Sort	500	4 878	203 526

4.3 Results — almost-sorted 100-element array

The 100–element array was first sorted into ascending order and then five pairs of neighbouring values were swapped (at indices 10/11, 25/26, 40/41, 60/61 and 80/81). All four algorithms were given the same almost–sorted array.

Algorithm	Input Size	Number of Comparisons	Execution Time (ns)
Selection Sort	100 (almost sorted)	4 950	99 104
Insertion Sort	100 (almost sorted)	104	4 198

Algorithm	Input Size	Number of Comparisons	Execution Time (ns)
Merge Sort	100 (almost sorted)	357	57 145
Quick Sort	100 (almost sorted)	4 735	106 823

Compared with the random 100–element array, Insertion Sort fell from 2 554 comparisons to 104 (a 96% reduction), Merge Sort fell slightly from 548 to 357, Selection Sort was unchanged at 4 950, and Quick Sort rose sharply from 705 to 4 735.

```

Part C - Sorting experiment results

=====
PART C - SORTING ALGORITHM EXPERIMENT
=====
Random values 1..999, fixed seed 2026. Each algorithm sorts an identical copy of the same array.

Algorithm      Input Size  Number of Comparisons  Execution Time (ns)
-----
Selection Sort  20          190                    7838
Insertion Sort  20           90                    4742
Merge Sort      20           68                    12513
Quick Sort      20           81                    6970
-----
Selection Sort  50          1225                   41309
Insertion Sort  50           652                   19921
Merge Sort      50           221                   31260
Quick Sort      50           271                   14948
-----
Selection Sort  100         4950                   110915
Insertion Sort  100         2554                   73273
Merge Sort      100          548                   64252
Quick Sort      100          705                   31174
-----
Selection Sort  500        124750                 2587601
Insertion Sort  500        61625                 1708888
Merge Sort      500        3861                  368981
Quick Sort      500        4878                  209758
-----

ADDITIONAL TEST - ALMOST-SORTED 100-ELEMENT ARRAY
The 100-element array is first sorted ascending, then five pairs of
neighbouring values are swapped. All four algorithms receive the same array.

Five neighbouring pairs swapped at indices 10/11, 25/26, 40/41, 60/61, 80/81.

Algorithm      Input Size  Number of Comparisons  Execution Time (ns)
-----
Selection Sort  100 (almost) 4950                    25338
Insertion Sort  100 (almost) 104                     4207
Merge Sort      100 (almost) 357                     25132
Quick Sort      100 (almost) 4735                    109710
-----

Note: measured times on the first run of a Java program are affected by
JIT warm-up, so the comparison counts are the more reliable measure.

```

Figure 7 — Part C: the experiment output produced by the program, including the almost-sorted test.

4.4 Answers to the Part C questions

1. Which algorithm performed the fewest comparisons?

Merge Sort, in almost every case. On the random arrays it needed 68 ($n=20$), 221 ($n=50$), 548 ($n=100$) and 3 861 ($n=500$) comparisons — the lowest at every size except $n=20$, where it was already lowest as well. On the almost-sorted array the fewest was Insertion Sort with only 104 comparisons, because that input is its best case.

2. Which algorithm performed the most comparisons?

Selection Sort, at every input size (190, 1 225, 4 950 and 124 750). Its comparison count is fixed at $n(n-1)/2$ regardless of how the data is arranged, so it can never do better than its worst case. On the almost-sorted array Quick Sort came close to it (4 735 versus 4 950) for the reason explained in question 3.

3. Which algorithm performed best on the almost-sorted array?

Insertion Sort, by a very wide margin: 104 comparisons and about 4 200 ns, against 357 for Merge Sort, 4 950 for Selection Sort and 4 735 for Quick Sort. Each key needs only one comparison when it is already larger than its predecessor, so the inner loop exits immediately and the algorithm approaches its $O(n)$ best case. Quick Sort, by contrast, degraded badly: with the last element as pivot, a nearly sorted array gives extremely unbalanced partitions, which drives it towards its $O(n^2)$ worst case.

4. Which algorithm appeared to scale best as the input became larger?

Merge Sort scaled best and most predictably. Going from 100 to 500 elements (five times the data), its comparisons grew about 7-fold (548 to 3 861), close to the $n \log n$ prediction, while Selection Sort grew 25-fold (4 950 to 124 750) and Insertion Sort 24-fold, both matching the n^2 prediction. Quick Sort scaled almost as well as Merge Sort on random data (705 to 4 878) and had the fastest measured time at $n=500$, but it is not as reliable because its performance depends on the pivot happening to split the data evenly.

5. Did the experimental results generally agree with the theoretical time complexity?

Yes. Selection Sort's counts were exactly $n(n-1)/2$ — 190, 1 225, 4 950 and 124 750 — confirming the $O(n^2)$ prediction in all cases. Insertion Sort averaged roughly $n^2/4$ comparisons on random data (2 554 for $n=100$) and collapsed to about n on the almost-sorted array, matching its $O(n^2)$ average and $O(n)$ best case. Merge Sort stayed near $n \log n$ in every test (3 861 against a predicted 4 483 for $n=500$), confirming $O(n \log n)$ in all cases. Quick Sort matched $O(n \log n)$ on random data but rose towards $O(n^2)$ on the almost-sorted array, which is the documented worst case for a last-element pivot. The one apparent disagreement is at $n=20$, where Merge Sort had the fewest comparisons but the slowest time (13 299 ns); that is the constant overhead of recursion and of allocating temporary arrays, which dominates on tiny inputs, plus JIT warm-up at the start of a run.

6. Is execution time alone enough to decide which algorithm is better?

No. Execution time is machine- and run-dependent: it is affected by CPU speed and load, by the Java JIT compiler warming up during the run, by garbage collection and by cache behaviour, which is why Merge Sort was slowest at $n=20$ despite doing the fewest comparisons. Comparison counts are deterministic and reproducible, so they show the algorithm's actual work. A sound choice also weighs other factors: memory use (Merge Sort needs $O(n)$ extra space, the other three sort in place), stability (Insertion and Merge Sort are stable, Selection and Quick Sort are not), worst-case guarantees (Merge Sort guarantees $O(n \log n)$; Quick Sort does not), behaviour on the data actually expected, and code simplicity. Time should be read together with comparison counts and complexity analysis, never on its own.

5. Part D — Integrated Service-Centre System

The queue, the singly linked list, the array statistics and the four sorting algorithms all run inside one program, `ServiceCentreApp.java`. The four students from the brief are pre-loaded at start-up so every option can be exercised immediately. Serving a student (option 2) also appends that student's service time to the statistics array, which is what option 8 reports on and option 9 sorts — so the components genuinely interoperate rather than merely coexisting. The postfix stack exercise remains separate, as the brief requires.

Menu option	Structure / operation invoked	How it works
1. Add student to waiting queue	Queue — enqueue	A new node is linked behind the rear pointer; $O(1)$.
2. Serve next student	Queue — peek + dequeue	<code>peek()</code> shows who is next, <code>dequeue()</code> advances the front pointer, and the service time is recorded in the array.
3. Display waiting students	Queue — traversal	Walks from front to rear printing each node, and draws the queue diagram.
4. Add student service record	Linked list — <code>insertStudent()</code>	Inserts at the beginning, at the end or at a chosen position by re-pointing next references.
5. Display student service records	Linked list — traversal	Follows next from the head to NULL, printing each node.
6. Search for student record	Linked list — search	Sequential search by student number; reports the node position and the number of comparisons.
7. Remove student record	Linked list — deletion	Finds the predecessor and points it past the target node; prints the list before and after.
8. Display daily statistics	Array processing	One traversal of the service-time array computes all six statistics.
9. Sort service times	Selection / Insertion / Merge / Quick	Copies the service times out of the linked list into an array and sorts the chosen way, reporting comparisons, movements and time.
10. Run sorting experiment	Sorting experiment	Runs the complete Part C experiment, including the almost-sorted test.
11. Exit	—	Ends the program.

```

Option 3 - Display waiting students (Queue traversal)

=====
CAMPUS SERVICE CENTRE
=====

1. Add student to waiting queue
2. Serve next student (remove from queue)
3. Display waiting students
4. Add student service record (Linked List - insertStudent())
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit
=====

Select option: 3
--- WAITING QUEUE (front to rear) ---
Pos  StudentNo  Name      Service      Est.Time
1    221045678  Maria     Registration  12 min
2    222034512  Tomas     Student Card  5 min
3    223041876  Ndapewa   Fees          8 min
4    221067341  Simon     Documents     4 min
Front of queue : Maria(221045678)
Rear of queue  : Simon(221067341)
Students waiting: 4
Queue diagram: FRONT -> [Maria] -> [Tomas] -> [Ndapewa] -> [Simon] -> REAR

```

Figure 8 — the integrated menu, and option 3 displaying the waiting queue by traversal.

```

Options 4 and 5 - Linked list insertion and traversal

Select option: 4
Select option:
Student number: Name      : Service type : Estimated service time (minutes): Insert where? 1 = beginning 2 = end 3 = at a position
Choice: Record added. List diagram: HEAD -> [Kaleb | next] -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> [Simon | next] -> NULL

Select option: 5
--- STUDENT SERVICE RECORDS (linked list, head to tail) ---
Node StudentNo  Name      Service      Est.Time
1    226044321  Kaleb     Student Card  6 min
2    221045678  Maria     Registration  12 min
3    222034512  Tomas     Student Card  5 min
4    223041876  Ndapewa   Fees          8 min
5    221067341  Simon     Documents     4 min
Total records: 5
List diagram: HEAD -> [Kaleb | next] -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> [Simon | next] -> NULL

```

Figure 9 — options 4 and 5: a record is inserted at the beginning of the linked list, then the whole list is traversed.

```

Options 6 and 7 - Linked list search and deletion

Select option: 6
Enter the student number to search for: Found at node position 4 after 4 comparison(s).
StudentNo  Name      Service      Est.Time
223041876  Ndapewa   Fees          8 min

Select option: 7
Enter the student number to remove: List before deletion: HEAD -> [Kaleb | next] -> [Maria | next] -> [Tomas | next] -> [Ndapewa | next] -> [Simon | next] -> NULL
Record 222034512 removed.
List after deletion : HEAD -> [Kaleb | next] -> [Maria | next] -> [Ndapewa | next] -> [Simon | next] -> NULL

```

Figure 10 — options 6 and 7: searching for student 223041876 (found at node 4 after 4 comparisons) and deleting record 222034512, with the list shown before and after.

```
Option 9 - Sorting the service times

Select option: 9
Service times taken from the linked list: [6, 12, 8, 4]
Choose an algorithm: 1 = Selection 2 = Insertion 3 = Merge 4 = Quick
Choice: Sorted ascending: [4, 6, 8, 12]
Merge Sort      comparisons=5      movements=8      time=469405 ns
```

Figure 11 — option 9: the service times held in the linked list are sorted with Merge Sort, with comparisons, movements and execution time reported.

6. Part E — Pseudocode and Algorithm Representation

The pseudocode below corresponds directly to the submitted Java implementation; the file and method that implements each algorithm is named in the heading. The complete source code is in the `src/` folder of the submission and is not reproduced in this report.

6.1 Queue

`enqueue(student)` — `StudentQueue.enqueue()`

```
ALGORITHM enqueue(student)
1. newNode <- new QNode holding student; newNode.next <- NULL
2. IF front = NULL THEN // the queue is empty
3.   front <- newNode; rear <- newNode
4. ELSE
5.   rear.next <- newNode // link behind the current last node
6.   rear <- newNode // the new node becomes the rear
7. END IF
8. size <- size + 1
END                                     Time complexity: O(1)
```

`dequeue()` — `StudentQueue.dequeue()`

```
ALGORITHM dequeue()
1. IF front = NULL THEN PRINT "queue is empty"; RETURN NULL
2. served <- front.data
3. front <- front.next // move the front pointer forward
4. IF front = NULL THEN rear <- NULL // the queue has just become empty
5. size <- size - 1
6. RETURN served
END                                     Time complexity: O(1)
```

6.2 Stack and postfix evaluation

`push(value)` and `pop()` — `IntStack`

```
ALGORITHM push(value)
1. IF top = capacity - 1 THEN // the backing array is full
2.   bigger <- new array of size capacity * 2
3.   FOR i <- 0 TO capacity - 1 DO bigger[i] <- items[i] END FOR
4.   items <- bigger
5. END IF
6. top <- top + 1; items[top] <- value
END

ALGORITHM pop()
1. IF top = -1 THEN ERROR "stack underflow"
2. value <- items[top]; top <- top - 1
3. RETURN value
END

ALGORITHM peek()
1. IF top = -1 THEN ERROR "stack is empty"
2. RETURN items[top]
END
```

`evaluatePostfix(expression)` — `PostfixEvaluator.evaluate()`

```

ALGORITHM evaluatePostfix(expression)
1.  CREATE empty stack S
2.  tokens <- split expression on spaces
3.  FOR EACH token IN tokens DO
4.      IF token is a number THEN
5.          S.push(token as number)
6.      ELSE                                     // token is +, -, * or /
7.          IF S.size < 2 THEN ERROR "malformed expression"
8.          right <- S.pop()                     // the second operand is popped first
9.          left  <- S.pop()
10.         S.push( apply(left, token, right) )
11.     END IF
12. END FOR
13. IF S.size <> 1 THEN ERROR "malformed expression"
14. answer <- S.peek()
15. RETURN S.pop()
END

```

Time complexity: $O(n)$ tokens

6.3 Singly linked list

insertNode — beginning, end and specified position

```
ALGORITHM insertAtBeginning(student)
1. newNode <- new Node holding student
2. newNode.next <- head           // point at the old first node
3. head <- newNode
4. count <- count + 1
END                                O(1)

ALGORITHM insertAtEnd(student)
1. newNode <- new Node holding student
2. IF head = NULL THEN head <- newNode
3. ELSE
4.     current <- head
5.     WHILE current.next <> NULL DO current <- current.next END WHILE
6.     current.next <- newNode
7. END IF
8. count <- count + 1
END                                O(n)

ALGORITHM insertAtPosition(student, position) // position is 1-based
1. IF position < 1 OR position > count + 1 THEN PRINT "invalid"; RETURN FALSE
2. IF position = 1 THEN insertAtBeginning(student); RETURN TRUE
3. newNode <- new Node holding student
4. current <- head
5. FOR i <- 1 TO position - 2 DO current <- current.next END FOR
6. newNode.next <- current.next // link to the rest of the list
7. current.next <- newNode     // link the predecessor to the new node
8. count <- count + 1; RETURN TRUE
END                                O(n)
```

deleteNode(studentNo) — ServiceRecordList.deleteStudent()

```
ALGORITHM deleteNode(studentNo)
1. IF head = NULL THEN PRINT "list is empty"; RETURN FALSE
2. IF head.data.studentNo = studentNo THEN // deleting the head
3.     head <- head.next; count <- count - 1; RETURN TRUE
4. END IF
5. previous <- head; current <- head.next
6. WHILE current <> NULL DO
7.     IF current.data.studentNo = studentNo THEN
8.         previous.next <- current.next // bypass the target node
9.         count <- count - 1; RETURN TRUE
10.    END IF
11.    previous <- current; current <- current.next
12. END WHILE
13. PRINT "student not found"; RETURN FALSE
END                                O(n)
```

searchNode(studentNo) and traverseList()

```

ALGORITHM searchNode(studentNo)           // sequential (linear) search
1. current <- head; position <- 1
2. WHILE current <> NULL DO
3.     IF current.data.studentNo = studentNo THEN
4.         PRINT "found at node", position; RETURN current.data
5.     END IF
6.     current <- current.next; position <- position + 1
7. END WHILE
8. PRINT "not found"; RETURN NULL
END                                         O(n)

ALGORITHM traverseList()
1. current <- head; position <- 1
2. WHILE current <> NULL DO
3.     PRINT position, current.data.studentNo, name, serviceType, estimatedTime
4.     current <- current.next; position <- position + 1
5. END WHILE
6. PRINT "total records =", count
END                                         O(n)

```

6.4 Array statistics — DailyStatistics

```
ALGORITHM computeDailyStatistics(serviceTimes, n)
1. total <- 0; served <- 0; longerThanTen <- 0
2. highest <- serviceTimes[0]; lowest <- serviceTimes[0]
3. FOR i <- 0 TO n - 1 DO
4.     served <- served + 1
5.     total <- total + serviceTimes[i]
6.     IF serviceTimes[i] > highest THEN highest <- serviceTimes[i]
7.     IF serviceTimes[i] < lowest THEN lowest <- serviceTimes[i]
8.     IF serviceTimes[i] > 10 THEN longerThanTen <- longerThanTen + 1
9. END FOR
10. average <- total / served
11. RETURN served, total, average, highest, lowest, longerThanTen
END
One traversal,  $O(n)$ . No built-in max, min or sum is used.
```

6.5 Sorting

selectionSort(A) — Sorters.selectionSort()

```
ALGORITHM selectionSort(A, n)
1. FOR i <- 0 TO n - 2 DO
2.     minIndex <- i
3.     FOR j <- i + 1 TO n - 1 DO
4.         comparisons <- comparisons + 1
5.         IF A[j] < A[minIndex] THEN minIndex <- j
6.     END FOR
7.     IF minIndex <> i THEN
8.         SWAP A[i] WITH A[minIndex]; swaps <- swaps + 1
9.     END IF
10. END FOR
END
Comparisons always  $n(n-1)/2$ ; best = average = worst =  $O(n^2)$ 
```

insertionSort(A) — Sorters.insertionSort()

```
ALGORITHM insertionSort(A, n)
1. FOR i <- 1 TO n - 1 DO
2.     key <- A[i]; j <- i - 1
3.     WHILE j >= 0 DO
4.         comparisons <- comparisons + 1
5.         IF A[j] > key THEN
6.             A[j + 1] <- A[j] // shift the larger value right
7.             shifts <- shifts + 1; j <- j - 1
8.         ELSE
9.             EXIT WHILE // correct position found
10.        END IF
11.    END WHILE
12.    A[j + 1] <- key
13. END FOR
END
Best  $O(n)$  on (almost) sorted data; average and worst  $O(n^2)$ 
```

mergeSort(A, low, high) — Sorters.mergeSort()

```

ALGORITHM mergeSort(A, low, high)
1. IF low >= high THEN RETURN           // BASE CASE: 0 or 1 element is sorted
2. mid <- low + (high - low) / 2
3. mergeSort(A, low, mid)               // divide the left half
4. mergeSort(A, mid + 1, high)          // divide the right half
5. merge(A, low, mid, high)             // conquer: combine the sorted halves
END

ALGORITHM merge(A, low, mid, high)
1. L <- copy of A[low .. mid]; R <- copy of A[mid + 1 .. high]
2. i <- 0; j <- 0; k <- low
3. WHILE i < length(L) AND j < length(R) DO
4.     comparisons <- comparisons + 1
5.     IF L[i] <= R[j] THEN A[k] <- L[i]; i <- i + 1
6.     ELSE A[k] <- R[j]; j <- j + 1
7.     END IF
8.     k <- k + 1
9. END WHILE
10. WHILE i < length(L) DO A[k] <- L[i]; i <- i + 1; k <- k + 1 END WHILE
11. WHILE j < length(R) DO A[k] <- R[j]; j <- j + 1; k <- k + 1 END WHILE
END
    O(n log n) in every case; needs O(n) extra memory

```

quickSort(A, low, high) — Sorters.quickSort()

```

ALGORITHM quickSort(A, low, high)
1. IF low >= high THEN RETURN           // BASE CASE: 0 or 1 element
2. p <- partition(A, low, high)
3. quickSort(A, low, p - 1)             // left partition (values <= pivot)
4. quickSort(A, p + 1, high)            // right partition (values > pivot)
END

ALGORITHM partition(A, low, high)       // Lomuto scheme
PIVOT RULE: the LAST element of the sub-array, A[high]
1. pivot <- A[high]; i <- low - 1
2. FOR j <- low TO high - 1 DO
3.     comparisons <- comparisons + 1
4.     IF A[j] <= pivot THEN
5.         i <- i + 1
6.         IF i <> j THEN SWAP A[i] WITH A[j]; swaps <- swaps + 1
7.     END IF
8. END FOR
9. SWAP A[i + 1] WITH A[high]           // pivot into its final position
10. swaps <- swaps + 1
11. RETURN i + 1
END
    Average O(n log n); worst O(n^2) on (almost) sorted input

```

7. Conclusion

The project shows that the choice of a data structure or algorithm follows from the operations a task actually performs. A queue enforces first-come-first-served service at $O(1)$ per student; a singly linked list absorbs insertions and deletions in the middle of the service records without shifting data; a stack evaluates postfix expressions in one pass because the operands needed are always the most recent values; and an array gives the contiguous, index-addressable storage that statistics and in-place sorting require.

The experiment confirmed the theory taught in class. Selection Sort's comparison count was exactly $n(n-1)/2$ at every size and never benefited from ordered data. Insertion Sort matched $O(n^2)$ on random data but collapsed to near $O(n)$ on the almost-sorted array. Merge Sort held to $O(n \log n)$ in every test and scaled best, at the cost of $O(n)$ extra memory. Quick Sort was the fastest on large random arrays but degraded towards $O(n^2)$ on almost-sorted input because of the last-element pivot rule — a concrete reminder that an algorithm's average case is not a guarantee, and that execution time on its own is not a sufficient basis for choosing one.

Repository and submission

Item	Detail
GitHub repository	https://github.com/georgeshimaneni23-beep/DSA521S-Group11-ServiceCentre
eLearning submission	DSA521S_Group11_Project2026.zip (report PDF, all source files, README)
Submitted by	225018497 – George Shimaneni
Contents of the repository	src/ Java source files, README.md, PSEUDOCODE.md, screenshots/, captured program output, and this report

The repository history records the progressive development of the solution as a series of commits, each describing the component added at that step, and every group member commits under their own GitHub account. Every member understands the complete submitted work and can explain, trace and defend any data structure, algorithm, pseudocode listing or section of source code in it.